All Labs Companion

The SQL and PostgreSQL toolkit behind Labs 1–5, the class demos, and the hands-on half of the midterm

Advanced Data Management

Covers the Chapter 1 and 4 class examples, Labs 3, 4 and 5, and Part 3 of the mock midterm

How to read this companion

The lecture chapters explain *why* a plan costs what it costs. The labs, and Part 3 of the midterm, ask you to *type* things: generate a skewed table with one statement, write a group-by with a date filter, create the right kind of index, force a join algorithm, read a `pg_stats` row, verify a rows-per-page figure with `pageinspect`. None of that is hard, but all of it is easy to forget if you have not touched SQL for a while. This document rebuilds the toolkit from the ground up, in the order you need it: first the SQL that every lab is written in (Part I), then the PostgreSQL instruments that let you watch the engine (Part II), then each lab walked through command by command with what you should see and what the submitted exercises want (Part III), and finally a write-from-memory drill whose items are the midterm's hands-on questions in disguise, with full solutions (Part IV). A one-page cheat sheet closes it. Every command here runs on the `adm_lab` database from Lab 1, or on the tiny fallback schema of §8, which you can create in thirty seconds on any PostgreSQL.

Part I

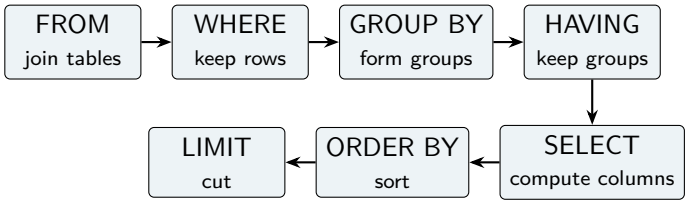
SQL, rebuilt in one sitting

1 The shape of a query and the order it runs in

Every query in the labs is an instance of one shape. Written top to bottom, evaluated in a different order:

```
SELECT   c.country, count(*) AS n, round(avg(o.amount), 2) AS avg_amt  -- 5
FROM     customers c JOIN orders o ON o.cust_id = c.id                 -- 1
WHERE    o.order_date >= date '2025-06-01'                             -- 2
GROUP BY c.country                                                     -- 3
```

```
HAVING   count(*) > 1000                                    -- 4
ORDER BY n DESC                                             -- 6
LIMIT    5;                                                 -- 7
```



Three consequences you will hit in every lab:

- **WHERE filters rows, HAVING filters groups.** A condition on an aggregate (`count(*) > 1000`) can only live in `HAVING`, because before grouping there is no count to test.
- **Aliases.** `AS n` is usable in `ORDER BY n` (step 6 runs after step 5) but not in `WHERE n > 3` (step 2 runs before). Repeat the expression or use a subquery.
- **Every non-aggregated SELECT column must be in GROUP BY.** `SELECT country, city, count(*) ... GROUP BY country` is an error: the engine has no single `city` to show for a group.

Key insight. SQL is written SELECT-first but evaluated FROM-first. An alias born in SELECT does not exist yet in WHERE, but it does exist in ORDER BY.

This order is also the order the executor's operators appear in a plan, read bottom-up: scans and joins (FROM/WHERE), then an aggregate (GROUP BY/HAVING), then a Sort and a Limit on top. Chapter 1 made the map; Lab 4 asks you to label it.

2 Joins

```
-- inner join: only pairs that match
SELECT c.name, o.amount
FROM   customers c JOIN orders o ON o.cust_id = c.id;

-- left join: every customer, with NULL amount for those without orders
SELECT c.name, o.amount
FROM   customers c LEFT JOIN orders o ON o.cust_id = c.id;

-- customers with no orders at all (anti-join via LEFT JOIN ... IS NULL)
SELECT c.name
FROM   customers c LEFT JOIN orders o ON o.cust_id = c.id
WHERE  o.id IS NULL;
```

The `ON` clause says *what* matches; the engine decides *how* (nested loop, merge or hash; Chapter 4). Aliases (`c`, `o`) are not optional once two tables share column names like `id`. The midterm schema joins `events e` to `users u` on `e.user_id = u.user_id`; same shape, different names.

Warning / Gotcha: A filter on the right table turns a LEFT JOIN back into an inner join

`LEFT JOIN orders o ... WHERE o.amount > 100` drops the customers with no orders, because their `o.amount` is `NULL` and `NULL > 100` is not true. If you want "all customers, and their big orders if any", move the condition into the `ON`.

3 Grouping, aggregates and conditional counting

```
SELECT country, count(*) AS customers
FROM   customers
GROUP BY country
HAVING count(*) > 1800
ORDER BY customers DESC;
```

Aggregates: `count(*)`, `count(col)` (ignores NULLs), `sum`, `avg`, `min`, `max`. Two idioms the labs lean on:

Conditional aggregate with `FILTER`. Lab 5 computes a selectivity by hand as the fraction of rows satisfying a predicate:

```
SELECT count(*) FILTER (WHERE country = 'MA')::float / count(*)
FROM   customers;                                    -- e.g. 0.2003
```

`count(*) FILTER (WHERE p)` counts only the rows where `p` holds. The older spelling is `sum(CASE WHEN p THEN 1 ELSE 0 END)`.

Warning / Gotcha: Integer division silently gives 0

Both counts are integers, and in PostgreSQL `2003 / 10000` is `0`. The `::float` cast on the numerator is what makes the division real. Forgetting it is the most common reason a by-hand selectivity comes out as exactly zero.

Per-day or per-month buckets. Cast or truncate the timestamp, then group by the result:

```
SELECT ts::date AS day, count(*)            -- or date_trunc('day', ts)
FROM   events
GROUP BY day ORDER BY day;
```

4 Subqueries and CASE

```
-- IN subquery: customers who ever placed an order above 100
SELECT name FROM customers
WHERE  id IN (SELECT cust_id FROM orders WHERE amount > 100);

-- EXISTS: same question, correlated form
SELECT name FROM customers c
```

```
WHERE  EXISTS (SELECT 1 FROM orders o WHERE o.cust_id = c.id AND o.amount >
    100);

-- subquery in FROM (a derived table): must have an alias
SELECT country, avg(total)
FROM  (SELECT c.country, c.id, sum(o.amount) AS total
       FROM customers c JOIN orders o ON o.cust_id = c.id
       GROUP BY c.country, c.id) t
GROUP BY country;
```

The rewriter (Chapter 5) turns `IN` and `EXISTS` into a semi-join, so do not expect the plan to contain a "subquery" node.

`CASE` is SQL's if/else expression, and it is the heart of the skewed-data generator in §7:

```
SELECT amount,
       CASE WHEN amount < 20  THEN 'small'
            WHEN amount < 80  THEN 'medium'
            ELSE                   'large' END AS band
FROM   orders;
```

Branches are tested in order; the first true `WHEN` wins, and `ELSE` catches the rest (without it the result is `NULL`).

5 Types, casts, dates and strings

The labs use a small set of types; knowing the casts between them removes most "operator does not exist" errors.

Type	Used for	Notes
`int`, `bigint`	ids, counts	`floor(x)::int` turns a double into an int
`serial`	auto-increment id	`id serial PRIMARY KEY`
`numeric(8,2)`	money	exact; `round(x::numeric, 2)`
`double precision`	`random()`, `avg`	`::float` forces real division
`text`, `char(2)`	names, country codes	`||` concatenates; `lower()`, `upper()`
`date`	`order_date`	`date + int` adds days
`timestamp`	`ts`, `signup_ts`	`timestamp - interval '30 days'`
`jsonb`	`payload`	`payload->>'key'` extracts text

```
SELECT '42'::int + 1;                          -- cast with ::  (PostgreSQL)
SELECT CAST('42' AS int) + 1;                  -- standard SQL spelling
SELECT 'Customer ' || 17;                      -- 'Customer 17' (int auto-cast
    to text)
SELECT date '2025-01-01' + 364;                -- 2025-12-31
SELECT now() - interval '30 days';             -- a timestamp 30 days ago
SELECT current_date - 30;                      -- a date 30 days ago
SELECT timestamp '2026-10-12' - random() * interval '365 days';  -- random
    moment in the past year
SELECT round(12.3456::numeric, 2);             -- 12.35 (round(double,int) does
     not exist!)
```

```
SELECT lower('Customer 7777');              -- 'customer 7777'
```

Warning / Gotcha: Three casts that bite

(1) `round(random()*100+5, 2)` fails: `random()` is double precision and two-argument `round` exists only for `numeric`, hence the `::numeric` in every lab generator. (2) `floor(random()*5)` is a double; as an array index or an id it must become `::int`. (3) `count(*)/count(*)` is integer division; cast one side to `::float`.

6 DDL and DML: tables, keys, inserts, updates

```
CREATE TABLE customers (
  id      serial PRIMARY KEY,
  name    text,
  country char(2)
);
CREATE TABLE orders (
  id         serial PRIMARY KEY,
  cust_id    int REFERENCES customers(id),     -- foreign key
  product_id int,
  amount     numeric(8,2),
  order_date date
);
ALTER TABLE customers ADD COLUMN city text;      -- Lab 5, part 5
UPDATE customers
SET    city = CASE WHEN country = 'MA' THEN 'Casablanca' ELSE 'Other' END;
DELETE FROM orders WHERE amount < 0;
DROP TABLE IF EXISTS tmp;
```

A `PRIMARY KEY` creates a unique B-tree index automatically (that is why the midterm's `shopstream` "only has the primary-key indexes" without anyone creating them). A `REFERENCES` clause creates *no* index on the referencing column; `orders.cust_id` is unindexed until Lab 3 adds one, which is exactly the situation of `events.user_id` in mock question 3.3(f).

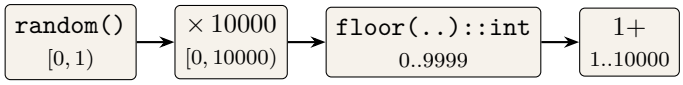
INSERT ... SELECT is how every lab fills a table: the `SELECT` manufactures rows, the `INSERT` stores them.

7 Generating data with `generate_series`

This is the single most reused piece of code in the course and the one the midterm asks you to write from memory (3.7b). Decode it once, piece by piece.

```
INSERT INTO orders (cust_id, product_id, amount, order_date)
SELECT 1 + floor(random() * 10000)::int,                  -- uniform id in
    1..10000
       1 + floor(random() * 200)::int,                    -- uniform id in
    1..200
```

```
       round((random() * 100 + 5)::numeric, 2),          -- uniform
    5.00..105.00
       date '2025-01-01' + floor(random() * 365)::int    -- uniform day of
    2025
FROM   generate_series(1, 50000) g;                      -- 50,000 rows
```



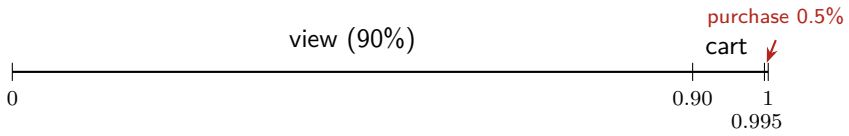
the same four steps give a date (add to a base date) or an array index

- `generate_series(1, n)` is a set-returning function: in `FROM` it behaves like a table of n rows with one column (here aliased `g`). The `SELECT` list is evaluated once per row, so every `random()` call draws afresh.
- `random()` is uniform on $[0, 1)$. Multiply by k, `floor`, cast, add 1: uniform integer in $1..k$. Multiply by a range width and add an offset: uniform real in `[5, 105)`.
- A **categorical** column comes from indexing an array literal (1-based):

```
INSERT INTO customers (name, country)
SELECT 'Customer ' || g,
       (ARRAY['MA','FR','ES','US','DE'])[1 + floor(random() * 5)::int]
FROM   generate_series(1, 10000) g;
ANALYZE;      -- always, after a bulk load (Part II explains why)
```

7.1 Skew: one draw, several cuts

Uniform data is the exception in real systems. To make `event_type` 90% `view`, 9.5% `cart`, 0.5% `purchase`, draw *one* number per row and cut the unit interval:




```
INSERT INTO events (user_id, event_type, ts, product_id, payload, device)
SELECT 1 + floor(random() * 2000000)::int,
       CASE WHEN r < 0.90  THEN 'view'
            WHEN r < 0.995 THEN 'cart'
            ELSE                'purchase' END,
       timestamp '2026-10-12' - random() * interval '365 days',
       1 + floor(random() * 50000)::int,
       '{}'::jsonb,
       (ARRAY['web','ios','android'])[1 + floor(random() * 3)::int]
FROM  (SELECT random() AS r FROM generate_series(1, 1000000)) g;
```

Warning / Gotcha: Why the subquery, and why not `random()` inside each WHEN

`CASE WHEN random() < 0.90 ... WHEN random() < 0.995 ...` draws a *new* number in the second branch. A row reaches the second test with probability 0.10, and then passes it with probability 0.995, so cart $\approx 9.95\%$ and purchase $\approx 0.10 \times 0.005 = 0.05\%$, ten times too rare. Naming the draw `r` in a derived table fixes it: every `WHEN` compares the same number. (Writing `random() AS r` directly in the outer `SELECT` does not work either: an alias cannot be referenced in the same `SELECT` list.)

8 The two schemas you will be asked about

`adm_lab` (labs)	Columns	Size
`customers`	`id PK, name, country char(2)` [+ `city` after Lab 5]	10,000–20,000 rows
`orders`	`id PK, cust_id FK, product_id, amount numeric(8,2), order_date`	50,000, grown to ≈500,000 in Lab 3
`products`	`id PK, ...`	200
`shopstream` (midterm)		
`users`	`user_id PK, email, country, city, segment, signup_ts`	2,000,000 rows, 30,000 pages
`products`	`product_id PK, title, price, category`	50,000 rows
`events`	`event_id PK, user_id, event_type, ts, product_id, payload jsonb, device`	50,000,000 rows, 1,000,000 pages, 50 rows/page

In `adm_lab`, `amount` is uniform on 5–105 (so `amount > 90` keeps about 15% and `> 104` about 1%), `order_date` is uniform over 2025, and `country` is uniform over five (fallback) or about a dozen (full Lab 1) values. In `shopstream`, `event_type` is skewed 90/9.5/0.5, `city` determines `country`, and statistics are stale by a factor of ten. Every hands-on midterm question is one of the lab experiments replayed on the bigger schema.

Part II

The PostgreSQL instruments

9 psql survival kit

```
psql -d adm_lab              # connect (or: psql -h host -U user -d adm_lab)
\timing on                   # print elapsed time after each statement
\d orders                    # columns, types, indexes of a table
\di                          # list indexes;  \dt  lists tables
\x                           # toggle expanded (vertical) output: use for
    pg_stats rows
```

```
\i lab3.sql                  # run a file;   \o out.txt   redirects output
\q                           # quit
```

The lab deliverables are Jupyter notebooks; there, `%load_ext sql` then `%sql postgresql://user:pw@localhost/adm_lab` (ipython-sql) lets a cell start with `%%sql` and hold the same statements. `\timing` has no notebook equivalent; use `EXPLAIN ANALYZE`'s `Execution Time` line instead, which is what you should report anyway.

10 EXPLAIN, line by line

```
EXPLAIN SELECT ...;                          -- plan + estimates only, does not
    run the query
EXPLAIN ANALYZE SELECT ...;                  -- runs it, adds actual rows and
    times
EXPLAIN (ANALYZE, BUFFERS) SELECT ...;       -- also page counts per node
EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON) SELECT ...;   -- machine-readable
```

Warning / Gotcha: EXPLAIN ANALYZE executes the statement

On an `INSERT`, `UPDATE` or `DELETE` it really modifies the table. Wrap it: `BEGIN; EXPLAIN ANALYZE DELETE ...; ROLLBACK;`

Here is the Lab 4 Part 1 plan with every field named:

```
Sort  (cost=1031..1036 rows=1996 width=40)
      (actual time=121.84..122.07 rows=1996 loops=1)
  Sort Key: c.name
  Sort Method: quicksort  Memory: 205kB
  Buffers: shared hit=3355
  ->  HashAggregate  (actual time=120.93..121.31 rows=1996 loops=1)
        Group Key: c.name
        Batches: 1  Memory Usage: 721kB
        ->  Hash Join  (actual time=4.02..105.66 rows=50112 loops=1)
              Hash Cond: (o.cust_id = c.id)
              ->  Seq Scan on orders o  (actual time=0.01..37.95 rows=500000 loops=1)
              ->  Hash  (actual time=3.91..3.92 rows=2000 loops=1)
                    Buckets: 2048  Batches: 1  Memory Usage: 117kB
                    ->  Seq Scan on customers c  (actual ... rows=2000 loops=1)
                          Filter: (country = 'MA'::bpchar)
                          Rows Removed by Filter: 18000
Planning Time: 0.412 ms
Execution Time: 122.38 ms
```

Field	Meaning
`cost=a..b`	planner's estimate in cost units (a sequential page read = 1): a to produce the first row, b to produce all. The root's b is what the optimizer minimised.
`rows=` (first pair)	**estimated** rows this node outputs.
`width=`	estimated average row size in bytes.
`actual time=a..b`	milliseconds to first row and to last row, *per loop*. A node's own work is roughly its b minus its children's b.
`rows=` (second pair)	**actual** rows per loop. Compare with the estimate: a gap of 10× or more is a statistics problem (Lab 5).
`loops=`	how many times the node ran. The inner side of a Nested Loop runs once per outer row; total rows = rows × loops.
`Buffers: shared hit=h read=r`	pages served from the buffer pool (h) and fetched from the OS/disk (r); $h + r$ is the page count the chapters' formulas predict.
`Filter` / `Rows Removed by Filter`	a predicate applied while scanning, and how many rows it discarded.
`Index Cond`	the part of the predicate the index itself answered.
`Sort Method`	`quicksort Memory: ...` (fit in `work_mem`) or `external merge Disk: ...` (Chapter 4's external sort).
`Buckets / Batches / Memory Usage`	hash table size; `Batches > 1` means the build side spilled: Grace hash join.

Technique: Reading a plan in four moves

1. **Find the leaves** (deepest indented): `Seq Scan`, `Index Scan`, `Bitmap Heap Scan`. Those are the tables and how they were read.
2. **Walk up**: each `->` is a child feeding the node above it. Joins have two children; the second child of a `Hash Join` is the `Hash` (build side), the first is the probe side.
3. **Compare estimate with actual at every node**, starting from the leaves. The first big mismatch is the root cause; everything above inherits it.
4. **Mark the pipeline breakers**: `Sort`, `Hash` (build), `HashAggregate`, `Materialize`. They must consume all input before producing output.

11 Steering the planner: SET and RESET

```
SET enable_seqscan   = off;   -- also: enable_indexscan, enable_bitmapscan,
SET enable_hashjoin  = off;   --        enable_mergejoin, enable_nestloop,
SET enable_mergejoin = off;   --        enable_hashagg, enable_sort
SET work_mem = '64kB';        -- per-sort / per-hash memory ('256MB' to be
    generous)
RESET work_mem;               -- one setting back to default
RESET ALL;                    -- everything back; run it between lab parts
SHOW work_mem;                -- what is it right now?
```

The `enable_*` switches do not forbid an operator; they add a huge penalty to it, so the planner avoids it when any alternative exists. A plan that "ignores" the setting has no alternative (a join with no index and no memory for a hash is still a nested loop). These are teaching tools: in production you fix statistics or indexes, you do not pin plans with `SET`.

12 The catalog: what the planner knows

```
-- row and page counts the planner believes (updated by ANALYZE / VACUUM)
SELECT relname, reltuples::bigint AS est_rows, relpages
FROM   pg_class WHERE relname IN ('customers', 'orders');

-- per-column statistics
SELECT attname, n_distinct, null_frac, most_common_vals, most_common_freqs,
       histogram_bounds, correlation
FROM   pg_stats WHERE tablename = 'orders';

-- when were statistics last refreshed, and how many rows live now?
SELECT relname, n_live_tup, last_analyze, last_autoanalyze
FROM   pg_stat_user_tables WHERE relname = 'orders';

-- which indexes exist, and are they ever used?
SELECT indexrelname, idx_scan, pg_size_pretty(pg_relation_size(indexrelid))
FROM   pg_stat_user_indexes WHERE relname = 'orders';
```

Concept: Reading a `pg_stats` row

- `n_distinct`: positive means an absolute count of distinct values; **negative** means a fraction of the row count (-1: every value unique, -0.5: about half as many distinct values as rows). Negative is used for columns whose distinct count grows with the table.
- `null_frac`: fraction of NULLs; the planner multiplies equality selectivities by $(1 - \text{null_frac})$.
- `most_common_vals` / `most_common_freqs`: the MCV list. An equality on a listed value uses its exact frequency; on an unlisted value it shares the leftover uniformly.
- `histogram_bounds`: equi-depth histogram of the non-MCV values; ranges are estimated by the fraction of buckets they cover.
- `correlation`: $-1..1$, how well physical order follows the column's order; near 1 makes index range scans (and BRIN) cheap because matching rows sit together.

The planner never counts rows at plan time; it only reads these numbers. If they are stale, every estimate built on them is stale (Lab 5, Part 4).

```
ANALYZE;                        -- refresh statistics for every table
ANALYZE orders;                 -- one table
VACUUM ANALYZE orders;          -- also reclaim dead rows and refresh the
    visibility map
ALTER TABLE events ALTER COLUMN event_type SET STATISTICS 1000;  -- bigger
    MCV/histogram
CREATE STATISTICS s_geo (dependencies) ON country, city FROM customers;  --
    then ANALYZE
```

Autovacuum runs `ANALYZE` by itself once about 10% of a table has changed, which is why Lab 5's stale-statistics effect must be observed *right after* the insert (or with `ALTER TABLE orders SET (autovacuum_enabled = false)` during the experiment).

13 Sizes, pages and what is inside them

```
SELECT pg_size_pretty(pg_relation_size('orders'));        -- heap only, e.g.
    35 MB
SELECT pg_relation_size('orders') / 8192 AS pages;         -- number of 8 KB
    pages = B
SELECT pg_size_pretty(pg_total_relation_size('orders')); -- heap + indexes +
     TOAST
SELECT pg_size_pretty(pg_indexes_size('orders'));         -- indexes only

CREATE EXTENSION IF NOT EXISTS pageinspect;
SELECT count(*) FROM heap_page_items(get_raw_page('orders', 0));   -- rows on
     page 0
SELECT lp, lp_off, lp_len, t_xmin
FROM   heap_page_items(get_raw_page('orders', 0)) LIMIT 5;        -- the slot
     array
SELECT * FROM page_header(get_raw_page('orders', 0));              -- free
    space: upper-lower
```

Rows per page measured two ways: `reltuples / relpages` from the catalog (an estimate), or `count(*)` over `heap_page_items` of a few pages (exact for those pages). Mock 3.2(c) asks for precisely these two commands. A row's physical address is visible as the system column `ctid`: `SELECT ctid, id FROM orders LIMIT 3` shows `(page, slot)` pairs.

14 The buffer pool, observed

```
CREATE EXTENSION IF NOT EXISTS pg_buffercache;
SELECT c.relname, count(*) AS buffers, pg_size_pretty(count(*) * 8192) AS
    size
FROM   pg_buffercache b
JOIN   pg_class c ON b.relfilenode = pg_relation_filenode(c.oid)
WHERE  b.reldatabase IN (0, (SELECT oid FROM pg_database
                             WHERE datname = current_database()))
GROUP BY c.relname ORDER BY buffers DESC LIMIT 10;

SHOW shared_buffers;                                    -- pool size, e.g.
    128MB = 16384 pages
SELECT blks_hit, blks_read,
       round(blks_hit::numeric / nullif(blks_hit + blks_read, 0), 3) AS
    hit_ratio
FROM   pg_stat_database WHERE datname = current_database();
```

`pg_buffercache` has one row per frame; joining to `pg_class` names the table each frame holds. Run the query, scan a big table, run it again: with PostgreSQL's ring buffer for large scans the table appears with a few dozen frames, not the whole pool. The `BUFFERS` option of `EXPLAIN` gives the same hit/read split for one query.

15 Export to Parquet and query it with DuckDB

PostgreSQL's own `COPY` writes text, CSV or its binary format, not Parquet. Two ways to get there:

```
-- (1) with the pg_parquet extension installed in PostgreSQL
COPY (SELECT ts FROM events) TO '/tmp/events_ts.parquet' (FORMAT parquet);

-- (2) from DuckDB, pulling straight out of PostgreSQL
INSTALL postgres; LOAD postgres;
ATTACH 'dbname=shopstream' AS pg (TYPE postgres);
COPY (SELECT ts FROM pg.events) TO '/tmp/events_ts.parquet' (FORMAT parquet);

-- then, in DuckDB, count rows per day
SELECT ts::date AS day, count(*) AS n
FROM   read_parquet('/tmp/events_ts.parquet')      -- or simply FROM '/tmp/
    events_ts.parquet'
GROUP BY day ORDER BY day;
```

The Chapter 1 demo did the same with `orders` and `SELECT AVG(amount) FROM 'orders.parquet'`. Why it is cheaper: the Parquet file holds only the exported column, compressed, so DuckDB reads a few hundred megabytes instead of the 8 GB heap whose rows carry a TOAST pointer, a `jsonb` and six other columns for every 8-byte timestamp.

Part III

The labs, part by part

16 Lab 1 and the Chapter 1 class examples

There is no separate Lab 1 sheet in your folder; the Chapter 1 class examples carry its content and build the database every later lab uses.

Technique: What each example shows

1. **Setup.** The two `CREATE TABLE`s and two `INSERT ... SELECT ... generate_series` of §7, then `ANALYZE`. Without the `ANALYZE` the first `EXPLAIN` shows row estimates based on nothing.
2. **Life of a query.** `EXPLAIN` on the join-and-group query shows `HashAggregate` over `Hash Join` over two `Seq Scan`s, the customers scan carrying `Filter: country = 'MA'`. Bottom-up: that is the physical plan from the lecture. `EXPLAIN (ANALYZE, BUFFERS)` adds actual rows and page counts; compare them with the estimates.
3. **Declarative versus procedural.** `EXPLAIN SELECT * FROM orders WHERE amount > 100` gives a Seq Scan (plan A). After `SET enable_seqscan = off` and `CREATE INDEX idx_amount ON orders(amount)` it gives an Index or Bitmap scan (plan B). The

SQL did not change; the strategy did. `RESET enable_seqscan; DROP INDEX idx_amount;` afterwards.

4. **OLTP versus OLAP shapes.** `WHERE id = 4275` touches one row through the primary-key index; the country aggregate sweeps everything. Same database, two workloads.
5. **Row versus column.** `COPY (SELECT * FROM orders) TO '/tmp/orders.parquet' (FORMAT parquet)` then `SELECT AVG(amount) FROM '/tmp/orders.parquet'` in DuckDB reads one column of the file.
6. **SQL refresher.** Top-N (`ORDER BY total DESC LIMIT 5`), `HAVING count(*) > 1800`, and an `IN` subquery. Predict each plan before running `EXPLAIN`: Hash Join + HashAggregate + Sort + Limit; Seq Scan + HashAggregate; a semi-join (`Hash Semi Join` or `Nested Loop Semi Join`).
7. **The catalog is data.** `pg_class.reltuples` and `pg_stats.n_distinct` are ordinary rows you can query, and exactly the numbers the optimizer reads in example 2.

17 Lab 3: Indexing, measuring the payoff and the cost

Technique: Part by part, with what you should see

0. **Grow `orders` to 500k rows** with the generator (`generate_series(1, 450000)`), then `ANALYZE`. Index effects are invisible on 50k rows that fit in a few hundred pages.
1. **Baseline.** Both queries are `Seq Scan`. Record `Execution Time` and `Buffers: shared hit=...`: the buffers number is B, the page count of the table, the Chapter 2 cost.
2. **Add indexes.** `CREATE INDEX idx_cust_name ON customers(name)` turns the point lookup into an `Index Scan using idx_cust_name` touching 3–4 buffers. `idx_orders_date` turns the ten-day range (about 3% of the year, roughly 14,000 rows) into a `Bitmap Heap Scan` over a `Bitmap Index Scan`: PostgreSQL collects the matching row ids, sorts them by page, then reads the heap pages once each. That is still "the index being used".
3. **Prefix rule.** With `idx_oc ON orders(cust_id, order_date)`: `WHERE cust_id = 42` uses it (leading column); `cust_id = 42 AND order_date > ...` uses it (both columns); `order_date > ...` alone does not (the tree is sorted by `cust_id` first, so dates are scattered). If the third query shows `idx_orders_date` instead of a Seq Scan, that is Part 2's index helping; drop it to see the point.
4. **Selectivity flip.** `amount` is uniform on 5–105: `> 104` keeps about 1%, `> 60` about 45%, `> 20` about 85%. Expect Index/Bitmap scan at 1% and Seq Scan at the other two. Bisect the threshold (try 100, 95, 90, 85...) until the plan changes and report that fraction, typi-

cally 5–20% depending on physical order and `random_page_cost`. The chapter's rule $s < 1/(4 \cdot \text{rows per page})$ gives the ballpark for random heap fetches; the bitmap scan pushes the flip higher.

5. **Expression index.** `WHERE lower(name) = 'customer 7777'` with only `idx_cust_name` present is a Seq Scan: the tree is ordered by `name`, not by `lower(name)`. After `CREATE INDEX idx_lower_name ON customers(lower(name))` it is an Index Scan. The expression in the query must match the index expression textually.
6. **Write cost.** Time the 100k-row insert with three indexes on `orders`, drop them (`DROP INDEX idx_orders_date, idx_oc, idx_amount;`), time it again. Expect the indexed insert to be roughly 2–4× slower: every row writes a heap page plus one leaf per index, and the leaves are at random positions.

Worked example: The six submitted exercises, answered

1. Speedup = seq-scan time / index-scan time; with 20k customers expect a few ms versus a fraction of a ms, and more telling, buffers ≈ 150 versus ≈ 4.
2. Queries 1 and 2 used `idx_oc`; query 3 did not: an index serves a predicate only on a leftmost prefix of its columns.
3. The flip fraction you measured (say "the planner switched to Seq Scan between `> 95` (10%) and `> 90` (15%)"). Justify with the random-versus-sequential page cost argument: beyond a few percent of rows, one random heap read per match exceeds one sequential pass.
4. Show the two plans: Seq Scan with the plain index, Index Scan with the expression index.
5. Report both times and the ratio; explain it as one extra leaf write per index per row.
6. Index `orders.cust_id` (every join and most lookups go through it; selective, 10,000 distinct values). Do not index `customers.country` (five values, 20% per value, every query on it returns far more than the flip fraction, so the planner will scan anyway).

Interface / API: Every CREATE INDEX form you may be asked for

```
CREATE INDEX idx_date ON orders (order_date);                    -- B-tree (
    default): =, <, BETWEEN, ORDER BY
CREATE UNIQUE INDEX idx_email ON users (email);                  -- unique
    lookup, enforces uniqueness
CREATE INDEX idx_oc ON orders (cust_id, order_date);             -- multi-
    column: leftmost prefix rule
CREATE INDEX idx_lower ON users (lower(email));                  --
    expression index
CREATE INDEX idx_purch ON events (ts) WHERE event_type = 'purchase';   --
     partial index
CREATE INDEX idx_cov ON orders (cust_id) INCLUDE (amount);       -- covering
    : enables Index Only Scan
```

```
CREATE INDEX idx_ts_brin ON events USING brin (ts);          -- BRIN:
    huge, append-ordered column
CREATE INDEX idx_h ON users USING hash (email);              -- hash:
    equality only, no ranges
CREATE INDEX CONCURRENTLY idx_uid ON events (user_id);       --
    production: no write lock
DROP INDEX idx_h;
```

18 Lab 4: Operators, joins, sorting, aggregation

Technique: Part by part, with what you should see

1. **Read a plan.** The annotated plan of §10. Bottom-up: `Seq Scan on customers` (filter MA, 2,000 kept, 18,000 removed) → `Hash` (build, 117 kB, 1 batch); `Seq Scan on orders` (500,000 rows, probe side) → `Hash Join` (50,112 rows) → `HashAggregate` (1,996 groups) → `Sort` (quicksort in memory). Pipeline breakers: the `Hash` build, `HashAggregate`, `Sort`.
2. **Force the three joins.** Default is a `Hash Join` (≈ 230 ms: each table read once, build side 20,000 customers in 1.3 MB). `SET enable_hashjoin = off` gives a `Merge Join` (≈ 430 ms: customers come sorted from the primary-key index, orders need an external sort, `Sort Method: external merge Disk: 8184kB`). `SET enable_mergejoin = off` as well gives a `Nested Loop` (≈ 640 ms: `Index Scan on orders` with `loops=20000`, one index descent per customer, about 25 rows each). Then `RESET ALL`.
3. **External sort.** `SET work_mem = '64kB'`: `Sort Method: external merge Disk: 8192kB`, about 390 ms. `SET work_mem = '256MB'`: `Sort Method: quicksort Memory: 71680kB`, about 190 ms. Same input, two algorithms; memory chose.
4. **Hash versus sort aggregation.** `GROUP BY country`: `HashAggregate`, 12 groups, 24 kB. `GROUP BY name`: `GroupAggregate` over a `Sort`, about 19,000 groups. Few groups fit in a tiny hash table; many groups make the planner prefer sorting (and a GroupAggregate streams, group by group).
5. **Size flips the join.** `WHERE c.id < 10`: `Nested Loop` with two index scans, 9 outer rows, 225 results, 0.26 ms. No filter: `Hash Join`, 500,000 rows, 230 ms. Only the row counts changed; the optimizer changed the algorithm. Chapter 5 in one experiment.

Worked example: The five submitted exercises, answered

1. Label as in Part 1; the pipeline breaker that matters is whichever consumes the big input: here the `Sort` and `HashAggregate` see only 50k and 2k rows, so the join's hash build (2,000 rows) is cheap and the plan streams the 500,000 orders through.

2. Hash < merge < nested loop (232 / 432 / 641 ms). Hash reads both inputs once and the build fits in memory. Merge must sort 500,000 orders first, externally. Nested loop pays 20,000 index descents and scattered heap reads.
3. `external merge Disk: 8192kB` versus `quicksort Memory: 71680kB`: when the input exceeds `work_mem` PostgreSQL writes sorted runs to disk and merges them (Chapter 4's $2B$ per pass); with enough memory it sorts in place in one pass.
4. `GROUP BY country` (HashAggregate) versus `GROUP BY name` (Sort + GroupAggregate): the hash table has one entry per group; 12 entries is nothing, 19,000 is still small but the planner's estimate of hash memory versus a sort tipped it. Say it in terms of the number of groups.
5. $B_R = 200$, $B_S = 2000$, $M = 22$: $\lceil 200/(22-2) \rceil = 10$ chunks of the outer; cost $= 200 + 10 \times 2000 = 20{,}200$ page reads. (Mock 1.5 is the same formula with 300, 3,000, 32: $300 + 10 \times 3000 = 30{,}300$.)

Debugging: If a plan refuses to cooperate

- **The forced setting is ignored.** Check `SHOW enable_hashjoin`; a stale session setting from an earlier part is the usual cause. `RESET ALL`.
- **No external sort.** Lower `work_mem` (64 kB is the minimum) or sort `orders` after growing it; 50k rows of `orders` sort in memory even at small settings.
- **Nested loop shows `Materialize` or a Seq Scan on the inner.** There is no index on `orders.cust_id`; create one (Lab 3) to get the index nested loop of the demo.
- **`Batches: 4` under Hash.** The build side outgrew `work_mem`: Grace hash join. Raise `work_mem` to see `Batches: 1`.

19 Lab 5: Optimization, estimates, statistics and bad plans

Technique: Part by part, with what you should see

1. **Estimated versus actual.** For `amount > 90` on uniform 5–105 data, the Seq Scan should show `rows=` about 15% of the table against an actual within a few percent of it. For the MA join, the customers leaf is estimated from the MCV list (exact to within sampling), and the Hash Join output is the join-size formula $|R||S|/\max(d_R, d_S)$: 2,000 customers × 500,000 orders / 10,000 distinct `cust_id` = 100,000 (or 50,000 with 20,000 customers), close to actual. Write a two-column table per node: `rows=` estimated, `actual rows`.
2. **Read the statistics.** `\x` first, then the `pg_stats` query. For `orders`: `amount` has `n_distinct` about 10,000 (10,001 possible cents values), no MCVs worth listing, a 100-bucket histogram from 5 to 105; `cust_id` about 10,000 distinct; `order_date` 365 distinct, flat.

3. **Selectivity by hand.** `count(*) FILTER (WHERE country = 'MA')::float / count(*)` gives about 0.20 with five countries. `EXPLAIN SELECT * FROM customers WHERE country = 'MA'` shows `rows=` $\approx 0.20 \times N$. They agree because `'MA'` is in the MCV list with its measured frequency.
4. **Stale statistics.** Insert 300,000 rows; run `EXPLAIN ANALYZE ... amount > 90` *immediately*. The planner still believes `reltuples` = 50,000 (or 500,000) and estimates 15% of that; actual is 15% of the new total, so the estimate is off by the growth factor (7× from 50k). After `ANALYZE orders` the estimate snaps back. If autovacuum beat you to it, the troubleshooting note applies: insert more, or disable autovacuum on the table for the experiment.
5. **Correlated columns.** After `city = 'Casablanca' iff country = 'MA'`, both predicates have selectivity 0.2, and the planner multiplies: $0.2 \times 0.2 = 0.04$, so `rows=` $\approx$ 400 of 10,000 while actual is 2,000 (a 5× under-estimate). `CREATE STATISTICS s_geo (dependencies) ON country, city FROM customers; ANALYZE customers;` records that `city` determines `country`; the estimate becomes $\approx$ 2,000.

Worked example: The five submitted exercises, answered

1. Table of estimated versus actual per node; "accurate" means within a factor of about 2 at every node. With fresh `ANALYZE` both queries are accurate.
2. Hand selectivity × row count versus `rows=`: they agree to within the sample's precision (the MCV frequency comes from a 30,000-row sample, so a percent or two off).
3. Before `ANALYZE`: estimate stuck at the old table size; after: correct. It changed because `ANALYZE` rewrote `reltuples`, `relpages` and the `amount` histogram; the planner never looks at the table itself.
4. 400 estimated / 2,000 actual before; about 2,000 / 2,000 after `CREATE STATISTICS`. The dependency statistic replaces the independence product with the measured functional dependency.
5. A leaf estimate feeds every node above it. A join's size is $|R||S|/\max(d_R, d_S)$, so a 5× error at a leaf is a 5× error at the join and compounds through the next join. Worse, the algorithm and build-side choices are made on the wrong number: a Nested Loop chosen for "40 rows" runs 200,000 times (mock 3.1). An error near the top only mis-prices the last operator, with no choices left to damage.

Part IV

Write-from-memory drill

20 The map from labs to the midterm

Mock question	What it asks you to type or read	Where you practised it
3.1 Diagnose a plan	find the worst estimate, name the causes, `ANALYZE` + `CREATE STATISTICS`, name the better join	Lab 5 parts 1, 4, 5; Lab 4 part 2
3.2 Storage, columnar	pages and seconds; `COPY ... parquet` + DuckDB; `pg_relation_size`, `pageinspect`	Chapter 1 demo 4; Chapter 2 §6
3.3 Index design	six `CREATE INDEX` with types; one that no index helps, with a selectivity	Lab 3 parts 2–5
3.4 Sort and join cost	runs, passes, I/Os; which join; build side vs `work_mem`	Lab 4 parts 2–3, paper exercise
3.5 Estimation failures	independence product vs true count; `CREATE STATISTICS`; view vs purchase	Lab 5 parts 3, 5
3.6 Buffer behaviour	hit ratio, ring buffer, `pg_buffercache` query	Chapter 1 demo 1 (`BUFFERS`)
3.7 Write the SQL	group-by with a 30-day filter; skewed `generate_series` insert	Chapter 1 setup; Lab 3 part 0; §7

21 Drill: write these without looking

Do them on paper first, then type them into `adm_lab` (D1–D3 build the fallback schema if you do not have the database). Solutions follow in §22.

D1. Create the tables `customers(id, name, country)` and `orders(id, cust_id, product_id, amount, order_date)`, with keys.

D2. Fill `customers` with 10,000 rows, names `Customer 1` .. `Customer 10000`, country uniform over MA, FR, ES, US, DE.

D3. Fill `orders` with 50,000 rows: random customer, random product in 1..200, amount uniform 5–105 with two decimals, random day of 2025. Refresh statistics.

D4. Insert 1,000,000 `events` with `event_type` 90% view, 9.5% cart, 0.5% purchase, a random timestamp in the last year, and a random user in 1..2,000,000. (Mock 3.7b.)

D5. Number of purchase events per country in the last 30 days, biggest first. (Mock 3.7a.)

D6. Three refresher queries: top-5 customers by total spend; countries with more than 1,800 customers; names of customers who ever ordered above 100.

D7. Compute the real selectivity of `country = 'MA'` and show the planner's estimate for the same predicate. (Lab 5.3.)

D8. Write the index for each pattern: unique `email` lookup; a `ts` range;

`event_type = 'purchase'`; `event_type = 'view'`; `lower(email) = $1`; the join on `user_id`. Name each type, and say which pattern no index helps and why, with a number. (Mock 3.3.)

D9. Force PostgreSQL to run the customers–orders join as hash, then merge, then nested loop, showing each plan, and put everything back. (Lab 4.2.)

D10. Make `SELECT * FROM orders ORDER BY amount` spill to disk, then not, and name the line that proves it. (Lab 4.3.)

D11. Verify empirically that `events` has about 50 rows per page: two commands. (Mock 3.2c.)

D12. The plan in mock 3.1 estimates 40 Casablanca users and finds 200,000, and 50 join rows against 1,240,000. Write the commands that fix it and say which join should appear afterwards.

D13. Which tables occupy the buffer pool right now, biggest first. (Mock 3.6c.)

D14. Export only `events.ts` to Parquet and count events per day in DuckDB. (Mock 3.2b.)

D15. Find the indexes on `events` that have never been used since statistics were reset, with their sizes. (The practical answer to "which indexes to keep" in mock 2.3.)

D16. Trace, by hand, the estimate the planner makes for `WHERE country = 'MA' AND city = 'Casablanca'` with sel(country) = 0.2, sel(city) = 0.1 on 2,000,000 users, and the true count if every Casablanca user is in MA. Then write the fix. (Mock 3.5a.)

22 Drill solutions

Solution: D1. Schema

```
CREATE TABLE customers (id serial PRIMARY KEY, name text, country char(2)
    );
CREATE TABLE orders (id serial PRIMARY KEY,
                     cust_id int REFERENCES customers(id),
                     product_id int, amount numeric(8,2), order_date date
    );
```

Solution: D2. Customers

```
INSERT INTO customers (name, country)
SELECT 'Customer ' || g,
       (ARRAY['MA','FR','ES','US','DE'])[1 + floor(random() * 5)::int]
FROM   generate_series(1, 10000) g;
```

Solution: D3. Orders

```
INSERT INTO orders (cust_id, product_id, amount, order_date)
SELECT 1 + floor(random() * 10000)::int,
```

```
       1 + floor(random() * 200)::int,
       round((random() * 100 + 5)::numeric, 2),
       date '2025-01-01' + floor(random() * 365)::int
FROM   generate_series(1, 50000);
ANALYZE;
```

Check: `SELECT min(amount), max(amount), min(order_date), max(order_date) FROM orders;`

Solution: D4. Skewed events

```
INSERT INTO events (user_id, event_type, ts, product_id, payload, device)
SELECT 1 + floor(random() * 2000000)::int,
       CASE WHEN r < 0.90 THEN 'view' WHEN r < 0.995 THEN 'cart'
            ELSE 'purchase' END,
       timestamp '2026-10-12' - random() * interval '365 days',
       1 + floor(random() * 50000)::int,
       '{}'::jsonb,
       (ARRAY['web','ios','android'])[1 + floor(random() * 3)::int]
FROM  (SELECT random() AS r FROM generate_series(1, 1000000)) g;
```

Check the skew: `SELECT event_type, count(*) FROM events GROUP BY 1;` (`GROUP BY 1` means "by the first select column"). Expect about 900,000 / 95,000 / 5,000.

Solution: D5. Purchases per country, last 30 days

```
SELECT u.country, count(*) AS purchases
FROM   events e JOIN users u ON u.user_id = e.user_id
WHERE  e.event_type = 'purchase'
  AND  e.ts >= now() - interval '30 days'      -- or date '2026-10-12' -
    30
GROUP BY u.country
ORDER BY purchases DESC;
```

Both predicates are on `events`, so the rewriter pushes them below the join; with the partial index of D8(c) the leaf becomes an index or bitmap scan over about $0.5\% \times 1/12$ of the table.

Solution: D6. Refreshers

```
SELECT c.name, sum(o.amount) AS total
FROM   customers c JOIN orders o ON o.cust_id = c.id
GROUP BY c.name ORDER BY total DESC LIMIT 5;

SELECT country, count(*) FROM customers
GROUP BY country HAVING count(*) > 1800;

SELECT name FROM customers
WHERE  id IN (SELECT cust_id FROM orders WHERE amount > 100);
```

Solution: D7. Selectivity by hand versus planner

```
SELECT count(*) FILTER (WHERE country = 'MA')::float / count(*) FROM
    customers;
EXPLAIN SELECT * FROM customers WHERE country = 'MA';   -- read rows= on
    the Seq Scan
SELECT most_common_vals, most_common_freqs FROM pg_stats
WHERE  tablename = 'customers' AND attname = 'country'; -- where the
    estimate came from
```

Estimate = MCV frequency of `'MA'` × `reltuples`; with five uniform countries about $0.2 \times 10{,}000 = 2{,}000$.

Solution: D8. Six index patterns

```
-- (a) unique point lookup: B-tree (hash would also do; B-tree is the
    default and unique-capable)
CREATE UNIQUE INDEX idx_users_email ON users (email);
-- (b) date range: B-tree keeps order; BRIN is the cheap alternative if
    ts is append-ordered
CREATE INDEX idx_events_ts ON events (ts);
-- (c) rare value (0.5% = 250,000 rows): partial index, tiny, used by
    every purchase query
CREATE INDEX idx_events_purchase ON events (ts) WHERE event_type = '
    purchase';
-- (d) event_type = 'view': NO index helps (see below)
-- (e) function in the predicate: expression index
CREATE INDEX idx_users_lower_email ON users (lower(email));
-- (f) join key with no index: B-tree on the foreign-key side
CREATE INDEX idx_events_user ON events (user_id);
```

(d) is the pattern no index helps: selectivity 0.9, so 45,000,000 of 50,000,000 rows, about 900,000 of the 1,000,000 pages. The index-versus-scan threshold is roughly $1/(4 \cdot \text{rows per page}) = 1/200 = 0.5\%$ of the rows; 90% is 180 times above it. The planner will scan, and it is right.

Solution: D9. Three joins on demand

```
EXPLAIN ANALYZE SELECT * FROM customers c JOIN orders o ON o.cust_id = c.
    id;  -- Hash Join
SET enable_hashjoin = off;
EXPLAIN ANALYZE SELECT * FROM customers c JOIN orders o ON o.cust_id = c.
    id;  -- Merge Join
SET enable_mergejoin = off;
EXPLAIN ANALYZE SELECT * FROM customers c JOIN orders o ON o.cust_id = c.
    id;  -- Nested Loop
RESET ALL;
```

Solution: D10. External sort on demand

```
SET work_mem = '64kB';
EXPLAIN ANALYZE SELECT * FROM orders ORDER BY amount;   -- Sort Method:
    external merge  Disk: ...
SET work_mem = '256MB';
```

```
EXPLAIN ANALYZE SELECT * FROM orders ORDER BY amount;   -- Sort Method:
    quicksort  Memory: ...
RESET work_mem;
```

Solution: D11. Rows per page, measured

```
SELECT pg_relation_size('events') / 8192 AS pages,
       (SELECT reltuples::bigint FROM pg_class WHERE relname = 'events')
    AS rows;
                                                          -- rows / pages
    ~ 50
CREATE EXTENSION IF NOT EXISTS pageinspect;
SELECT count(*) FROM heap_page_items(get_raw_page('events', 0));   -- ~50
     on page 0
```

The first is catalog arithmetic (50,000,000 / 1,000,000), the second looks at one real page. Check a few pages (`get_raw_page('events', 12345)`) because page 0 may be special.

Solution: D12. Repairing the 3.1 plan

```
ANALYZE users;                  -- fixes reltuples and the MCV for city
    (40 -> ~200,000)
ANALYZE events;                 -- stale by 10x: 5M rows believed, 50M
    real
CREATE STATISTICS s_geo (dependencies) ON country, city FROM users;
ANALYZE users;                  -- the new statistic is only filled by
    ANALYZE
CREATE INDEX idx_events_user ON events (user_id);        -- optional, for
     an index nested loop later
EXPLAIN (ANALYZE, BUFFERS) <the query>;
```

Afterwards the planner knows 200,000 Casablanca users and about 1.2 million purchase events, so it chooses a **Hash Join**: build the hash on the filtered users (about 3,000 pages), probe with one pass over events, instead of 200,000 nested-loop scans of a 1,000,000-page table.

Solution: D13. What is in the pool

```
CREATE EXTENSION IF NOT EXISTS pg_buffercache;
SELECT c.relname, count(*) AS buffers
FROM   pg_buffercache b
JOIN   pg_class c ON b.relfilenode = pg_relation_filenode(c.oid)
WHERE  b.reldatabase IN (0, (SELECT oid FROM pg_database
                             WHERE datname = current_database()))
GROUP BY c.relname ORDER BY buffers DESC LIMIT 10;
```

Solution: D14. One column to Parquet, counted in DuckDB

```
-- in psql with pg_parquet (or run the COPY from DuckDB's postgres
    extension)
COPY (SELECT ts FROM events) TO '/tmp/events_ts.parquet' (FORMAT parquet)
```

```
    ;
-- in DuckDB
SELECT ts::date AS day, count(*) AS n
FROM   read_parquet('/tmp/events_ts.parquet')
GROUP BY day ORDER BY day;
```

Cheaper because the file holds one 8-byte column, compressed (a few hundred MB), while the heap scan reads 8 GB of full rows including TOAST pointers and the jsonb payload headers.

Solution: D15. Unused indexes

```
SELECT indexrelname, idx_scan,
       pg_size_pretty(pg_relation_size(indexrelid)) AS size
FROM   pg_stat_user_indexes
WHERE  relname = 'events' AND idx_scan = 0
ORDER BY pg_relation_size(indexrelid) DESC;
```

Keep an index if it is scanned, serves a selective predicate, or enforces uniqueness; drop the rest, because each one costs a leaf write per inserted row and disk space, and the planner never picks an index for a predicate that returns far more than the flip fraction.

Solution: D16. Independence versus dependency

Independence: $0.2 \times 0.1 = 0.02$, so $0.02 \times 2{,}000{,}000 = 40{,}000$ rows estimated. Truth: every Casablanca user is in MA, so the conjunction is just `city = 'Casablanca'`: $0.1 \times 2{,}000{,}000 = 200{,}000$. The planner underestimates by $5\times$ because it assumed the two predicates were independent coins when one implies the other.

```
CREATE STATISTICS s_geo (dependencies) ON country, city FROM users;
ANALYZE users;
```

Summary: The companion in five lines

1. SQL runs FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT; that order explains every error you will make.
2. Data is manufactured by `INSERT ... SELECT ... FROM generate_series`; uniform values come from `random()`, skew from one draw cut by `CASE`.
3. `EXPLAIN (ANALYZE, BUFFERS)` is the instrument: compare `rows=` with actual at the leaves, read `Buffers` as pages, watch `Sort Method` and `Batches`.
4. The planner knows only `pg_class` and `pg_stats`; `ANALYZE` rewrites them, `CREATE STATISTICS` teaches it dependencies.
5. An index earns its place only below the flip selectivity; the right form (unique, multi-column, expression, partial, covering, BRIN) follows the predicate's shape.

Cheat sheet

Need	Command
n rows of anything	`FROM generate_series(1, n) g`
uniform int in $1..k$	`1 + floor(random() * k)::int`
uniform real in $[a, a+w)$, 2 dp	`round((random() * w + a)::numeric, 2)`
random day of a year	`date '2025-01-01' + floor(random() * 365)::int`
random moment in the last year	`timestamp '2026-10-12' - random() * interval '365 days'`
categorical uniform	`(ARRAY['a','b','c'])[1 + floor(random() * 3)::int]`
categorical skewed	`CASE WHEN r < .9 THEN ..  WHEN r < .995 THEN .. ELSE ..  END` with `r` from `(SELECT random() AS r FROM generate_series ...)`
last 30 days	`ts >= now() - interval '30 days'`
per-day buckets	`GROUP BY ts::date`
fraction of rows matching p	`count(*) FILTER (WHERE p)::float / count(*)`
plan with estimates	`EXPLAIN q;`
plan with actuals and pages	`EXPLAIN (ANALYZE, BUFFERS) q;`
force an algorithm away	`SET enable_hashjoin = off;` ... `RESET ALL;`
spill a sort	`SET work_mem = '64kB';`
refresh statistics	`ANALYZE t;`
column statistics	`SELECT * FROM pg_stats WHERE tablename = 't';`
believed row/page counts	`SELECT reltuples, relpages FROM pg_class WHERE relname = 't';`
correlated columns	`CREATE STATISTICS s (dependencies) ON a, b FROM t; ANALYZE t;`
pages of a table	`pg_relation_size('t') / 8192`
rows on a page	`count(*) FROM heap_page_items(get_raw_page('t', 0))`
who is in the pool	`pg_buffercache` joined to `pg_class` on `relfilenode`
unused indexes	`pg_stat_user_indexes WHERE idx_scan = 0`
column to Parquet	`COPY (SELECT c FROM t) TO 'f.parquet' (FORMAT parquet);`
B-tree / unique / multi-col	`CREATE [UNIQUE] INDEX i ON t (a[, b]);`
expression / partial / covering	`ON t (lower(a))` / `ON t (a) WHERE p` / `ON t (a) INCLUDE (b)`
BRIN / hash	`USING brin (ts)` / `USING hash (a)`